

Transcript

Module 12: RESTful API Development

Video 1: Module Overview

Recall that in web applications, the code for the presentation tier can be executed on either the server or client. The former is known as server-side HTML rendering and the latter is known as client-side HTML rendering. Also recall that dynamic web applications rely on data stored in a database.

For a client-side rendering web application running on a client, there are two approaches to obtain data from a database running on a server. The first approach is to call Application Programming Interface or API endpoints manifesting as RESTful web services running on a server to access the database. The second approach is to directly utilise cloud service API methods to access a database. The former approach is preferable for two reasons. First, dedicated API endpoints allow you to fully customise the business processing logic to your exact requirements. Second, as an AI solution increases in scale and scope, it will likely feature multiple frontend web and mobile applications. In this scenario, it is more productive and maintainable to write a single set of API endpoints and share them across the multiple frontend applications.

In this module, you will learn how to create effective API endpoints manifesting as RESTful web services using both JavaScript and Python. We will begin with some basic concepts on RESTful web services before discussing the implementation details. Let's get started.

Video 2: Overview of Service-Oriented Architecture (SOA) and Microservices Architecture

This video will explain to you the two main software architectures that are commonly used in the implementation of RESTful web services. These are the Service-Oriented Architecture or SOA and Microservices Architecture. But before we even jump into these two software architectures, let's look at the more fundamental Component-based Architecture and the underlying Object-oriented Programming, or OOP paradigm.

OOP is a preferred paradigm for building a large-scale software application as a collection of classes and objects. This approach promotes modularity, reusability and maintainability of the code. But when applying OOP to develop the backend of a software application, inevitably, there will be many resulting classes. Individual classes are often too detailed and specific, and the behaviour of each class is also narrow in scope. More importantly, when the frontend of the software application needs to integrate with the backend, the task becomes increasingly challenging with too many classes. Consequently, there are at least two problems. First, there is complex coupling between the frontend and backend. Second, any changes in the backend classes will affect the frontend application thus complicating maintenance.

This infographic depicts frontend and backend integration with object-oriented software engineering. Observe that the presentation tier frontend application needs to interact with six classes. Component-based software engineering emerges to address this frontend and backend integration problem. In this approach, the backend of a software application is composed of multiple software components. Each component is a higher-level abstraction of multiple classes. Each component is thus a standalone code module that encapsulates a set of related functionalities and/or data.

For example, a shopping cart component might abstract the customer, product and transaction classes to perform the checkout and payment processing seamlessly. The tangible benefit is the reduction in the number of backend software components that the frontend application needs to interact with. This infographic depicts frontend and backend integration with component-based software engineering. Observe that the presentation tier frontend application only needs to interact with two components, essentially halving the number of backend elements.

Although the component-based architecture brings about an important benefit, it also has its own limitations. More specifically, the frontend application communicates with the software components via native method invocations. This means that the frontend and backend must be developed with the same programming language and technology. Component-based Architecture is thus not suitable for a heterogeneous computing environment in which software applications are developed with diverse programming languages and technologies.

The Service-Oriented Architecture or SOA effectively mitigates the limitations of the Component-based Architecture by introducing the notion of software services. Software services are software components that communicate by exchanging messages via a standard protocol instead of invoking methods natively. You can think of software services as neutral and independent software components that can support frontend software applications developed in a different programming language and all technology. SOA is thus more suitable for a real-world heterogeneous computing environment.

In practice, software services are typically implemented as web services, which use a standard-based internet protocol for exchanging messages. The de facto standard that is adopted universally is the HyperText Transport Protocol or HTTP. This infographic depicts frontend and backend integration with SOA-based web services. You can imagine frontend software applications developed with any programming languages and technologies on the left-hand side, accessing or utilising common business application logic or functionalities that are provided by the backend web services on the right-hand side. When web services are developed and deployed collectively in a single software entity, this is also known as monolithic SOA. In recent years, the Microservices Architecture has emerged as a popular alternative to monolithic SOA. The Microservices Architecture structures the backend of a software application as a collection of more loosely coupled web services that are developed and deployed independently of each other. You can imagine each backend software entity as consisting of just one web service. This gives rise to the notion of a microservice. This approach offers several benefits such as parallel and autonomous development and maintenance.

For example, if a microservice is brought offline for maintenance, the other microservices constituting the majority of the backend can continue normal operation. But the development process and techniques of the Microservices Architecture are generally more complex and thus it is more suitable for really large software applications. This infographic provides a high-level overview of the difference between monolithic SOA and Microservices Architecture.

Although Microservices Architecture has some unique distinctions in its development process and techniques, the fundamental building block of web services is similar to monolithic SOA. Thus, this module will focus on developing web services using monolithic SOA for simplicity of discussion.

Video 3: Introduction to RESTful Web Services

Recall that web services are software services that exchange messages via the Hyper Text Transfer Protocol or HTTP. This video helps you to gain a deep understanding of various essential concepts of RESTful or Representational State Transfer web services.

Let's begin by comparing the similarities and differences between the two main types of web services, namely big web services and RESTful web services.

Big web services exchange messages in XML format using the Simple Object Access Protocol or SOAP. SOAP defines the architecture and formats of messages that are exchanged between software entities. Big web services require a machine-readable description of the operations offered by the service written using the Web Services Description Language or WSDL. Thus, you can think of WSDL as an XML artifact for defining interfaces. In comparison, a traditional software component defines its operations using a class interface. Putting everything together, a frontend software application that wants to interact with a big web service will need to follow the operations definition in the WSDL artifact and then send request messages and receive response messages, formatted using SOAP over HTTP.

In comparison, RESTful web services exchange messages as plain text formatted using either Extensible Markup Language or XML and JavaScript Object Notation or JSON. Thus, there's no need for a messaging protocol such as SOAP. RESTful web services define the operations offered by the service using HTTP verbs or methods.

The typical HTTP methods that are used include PUT, GET, POST and DELETE corresponding to the standard data manipulation operations of Create, Retrieve, Update and Delete or CRUD. This convention is not a universal standard but following this convention makes it easier for other software entities to consume or utilise your RESTful web services. An interface definition standard such as WSDL is not required. Putting everything together, a frontend software application that wants to interact with a RESTful web service will only need to utilise HTTP to send request messages and receive response messages. This infographic provides a pictorial overview of RESTful web services. Observe that only HTTP is required.

This table provides a comprehensive summary of the technical characteristics of the different types of software components and services. Observe that RESTful web services are not only highly interoperable with dependent software entities written in any programming languages and technologies but also impose the most minimal requirements on them. Essentially, any dependent software entities that need to consume or utilise a RESTful web service only requires HTTP.

For these reasons and more, RESTful web services are very popular for developing the backend of web applications as they exhibit many desirable properties such as lightweight, loose coupling, scalability and architectural simplicity. Indeed, mobile applications and Internet of Things applications that run on low power devices are especially suited for backend built with RESTful web services.

Let's move on to examine the core principles of RESTful web services. First, RESTful web services are identified using URL or Uniform Resource Locator. Recall that a typical web application serving HTML content or webpages also relies on URL. In a web application, the URL identifies the folder or subfolders and file, or equivalently the route, that returns a HTML web page. In a RESTful web service, the URL identifies an endpoint that returns data formatted in either XML or JSON. Second, RESTful web services utilise a uniform interface for manipulating data or resources using the standard HTTP methods of PUT, GET, POST and DELETE. Recall that a typical web application also relies on HTTP methods but usually only uses GET and POST. In comparison, RESTful web services use two additional methods of PUT and DELETE. Collectively, the four HTTP methods correspond to the standard CRUD data manipulation as mentioned earlier. Third, RESTful web services only

exchange self-descriptive messages via decoupled resources that are represented as plain text XML or JSON.

I will explain more about JSON in the next video. In comparison, the SOAP messages exchanged by big web services rely on a XML Schema Definition or XSD, which further adds a layer of complexity in the communication process. Fourth and lastly, RESTful request messages are stateless or self-contained. Every request is considered unique. A RESTful web service is unable to determine whether multiple requests are from the same client or different client. This is mainly because the HTTP protocol itself is stateless by design. However, stateful interactions may be achieved by exchanging state via various techniques such as URL rewriting, cookies or hidden form fields.

Now that you have an excellent understanding of RESTful web services, let's move on to find out more about JSON, which is the more popular of the two message formats, the other being XML.

Video 4: Introduction to JSON

JSON or JavaScript Object Notation is one of two commonly used plain text formats for RESTful web services to exchange messages or data. The other format being XML or Extensible Markup Language. Between the two formats, JSON is the more popular one for good reasons. This example depicts a list of data records typically exchanged with a RESTful web service formatted in both JSON on the left and XML on the right.

In this example, we have a list of book records. Observe that JSON is similar to XML to the extent that both are self-describing or human readable. The two formats also utilise a hierarchical structure or values within value. For example, bookId and bookName are contained within a book.

Both JSON and XML can be passed and used by many programming languages. JSON and XML formatted data can both be fetched using the XMLHttpRequest object, which is supported in most modern web browsers. Client-side rendering web applications running within web browsers rely on the XMLHttpRequest object to communicate with RESTful web services running on the server-side. However, between JSON and XML they also have several differences JSON is more concise as it does not use end tag and there is no angle brackets surrounding the attribute names. Consequently, JSON is faster to read and write. JSON also supports the array notation. JSON can be parsed by a standard JavaScript function whereas XML requires an XML parser. JSON is generally preferable over XML as JSON is easier to pass and readily convertible to JavaScript object.

Note that JavaScript is one of the most popular programming languages. Other than structurally similar to JavaScript object, it also helps that JSON is similar to the dictionary or dict data structure in Python, which is widely considered the language of AI. Overall, it is extremely easy to work with JSON in both JavaScript and Python.

In this example, the sample code demonstrates the passing of the JSON book records into both JavaScript object and Python dict. It also demonstrates the respective conversion back to JSON string. Observe the ease of manipulating JSON across both JavaScript and Python.

In summary, it is easy to use JSON to represent both input and output data. For a GET or retrieve operation, the output data can be returned as JSON string. For PUT and POST, or create and update, operations, the input data can also be passed in as JSON string too. We will revisit JSON in future videos on the implementation of RESTful web services with JavaScript and Python.

Video 5: Best Practices in RESTful API Design

Application Programming Interface or API endpoints play a critical role in enabling communication between frontend software applications and the backend.

Thus, RESTful web services must be thoughtfully designed and implemented. This video explains some good practices that you should take into consideration when creating your own RESTful web services.

At a high-level, well-designed API should exhibit three characteristics. First, API should be easy to read and work with. The resources to be manipulated and their associated operations should be intuitive to the developers. Naming your API endpoints appropriately with the corresponding HTTP methods goes a long way in enhancing the usability. Second, API should adopt a defensive or failsafe design to make it easy for developers to integrate into their own code and avoid incorrect or erroneous usage. It is therefore important to provide informative and erroneous feedback. For example, input data validation and business rules checking in conjunction with appropriate HTTP status codes and meaningful messages can go a long way in helping developers to consume your API productively and correctly. Third, API should be complete and concise. A complete API should enable developers to build full-fledged applications using the data exposed by your endpoints. Of course, Rome is not built in a day. API completeness take place over time, and it is more likely that you will incrementally build on top of existing API endpoints.

Maintaining a comprehensive documentation of your API is also a good practice to ease the learning curve of new and experienced developers alike. You can easily use API design tool to define your API in the Swagger specification, which can then be used to generate the corresponding documentation automatically.

Let's move on to discuss more about error handling in API endpoints. If you are familiar with the HTTP protocol, you would be aware that there are many status codes available and for good reasons. By returning a more specific status code rather than a generic one, it helps the developer consuming your endpoints to take appropriate follow-up runtime actions or to troubleshoot their own code during design time. This table shows some of the more commonly used HTTP status codes. Observe that for successful requests, you should be using a 200 series status code and there are at least three such codes for different purposes. For erroneous conditions, you should be using a 400 or 500 series code as appropriate.

In terms of optimising the performance of API endpoints, you need to broadly differentiate between read and write operations. For read operations, you should make provision to retrieve one or few records based on well-defined filtering criteria. Operations for retrieving records in large quantity, or worst still retrieving all records, should be avoided as the user likely only requires a small subset of records for immediate use. Well-designed retrieval operations should facilitate record pagination with an appropriate page size. In addition, when there are relationships, or associations among records or classes, you should avoid fetching unnecessary relationship attributes. In general, fetching of to-one relationships is fine but fetching of to-many relationships should be avoided unless the user really requires those relationship data. It is generally preferable to create separate API endpoints and operations for fetching relationship attributes.

For write operations such as create, update and delete, you should consider providing support for bulk write operations to avoid the need of making excessive number of API calls. For example, to delete multiple records, it should be possible to pass in a list of record identifiers in a single operation call instead of making multiple operation calls, each taking in only one identifier for deletion. It is also important to perform comprehensive input data validation and business rule

checking to ensure that the input data passed in from the user are correct and appropriate. If your API is opened to external parties, you should not assume that they will perform the validation or understand how to validate the data correctly.

Incorrect or inappropriate input data will lead to unnecessary and inefficient processing at the downstream node. For example, if a piece of input data is of an incorrect data type or length, your endpoint should not attempt to insert the erroneous data into the database, which would otherwise be a waste of processing resources.

Finally, it is important to secure your API endpoints from unauthorised users. Standard security techniques such as JSON web token, hashing of password and encryption of confidential data should be implemented according to your data security requirements. If your API endpoints would be opened to external applications, you should also guard against unauthorised applications with an API or application key mechanism. I hope you would find these practices to be useful when designing and implementing your own API endpoints.

Video 6: Creating RESTful API Endpoints with Expressjs

In this video, you will learn how to implement API endpoints as RESTful web services using JavaScript and Express.js. Recall that in an earlier video, we have discussed about RESTful principles such as resource identification using URL and the use of standard HTTP methods to define operations. Observe that creating RESTful web services is actually very similar to creating server-side rendering web applications.

The main difference is that the former returns data formatted as JSON or XML whereas the latter returns HTML content. Thus, a lot of the concepts and techniques that we have discussed in an earlier module on web application are equally applicable to RESTful web services. But there are also some pertinent differences that I will be highlighting.

This video assumes that all data manipulated in the RESTful web services are stored and retrieved to or from a database using the same set of techniques discussed previously. We will be using a database containing a table of product records as shown in this figure.

Let's begin with retrieve operations using the HTTP GET method. Recall that in Express.js, the `res` or response object provides many methods to send response to the client and terminate the request-response cycle. In a web application, we typically use the `res.send()` or `res.render()` methods to return HTML content. In the case of API endpoints, we use the `res.json()` method to return data formatted as JSON. In the addition, the `res.sendStatus()` method can be used to return an appropriate HTTP status code. By default, HTTP 200 is returned.

In this example, we defined two GET endpoints for the `/product` path. The first endpoint retrieves all products whereas the second endpoint retrieves only a single product matching the required `productId` passed in as a route parameter. Observe that we perform error handling for the second endpoint. In particular, if the required `productId` does not exist, we return HTTP 404 to inform the client that the required product record is not found.

Next, let's look at a create operation using the HTTP PUT method. In this example, we pass in the new product record in JSON format to be created in the body of the PUT request. We perform error handling to check that the new `productId` does not currently exist in the database. If it exists, we return HTTP 400 to indicate a bad request instead of HTTP 201, which is the successful creation of a new record.

Observe that the first time we send the request, it would return HTTP 201 but for the second time, it will return HTTP 400. Can you figure out why? The update operation using the HTTP POST method is very similar to the create operation to the extent that we pass in the product record in JSON format to be updated in the body of the post request.

Note that only the PUT and POST methods allow you to pass in data through the request body. We also perform error handling to check that the productId to be updated currently exists in the database. Otherwise, we return HTTP 400 to indicate a bad request instead of HTTP 200. Last but not least, let's try a delete operation using the HTTP DELETE method. In this example, we pass in the productId to be deleted as a query string parameter to the /product path. If it exists and we are able to delete the record, we return HTTP 200. Otherwise, we return HTTP 400 to indicate a bad request.

Observe that the first time we send the request, it would return HTTP 200 but for the second time, it will return HTTP 400. Can you figure out why? As you would have observed by now, creating API endpoints and RESTful web services in JavaScript using Express.js is an easy task. Using the database integration capability also allows us to directly manipulate data in the database using the corresponding SQL statements.

I would strongly encourage you to write additional endpoints to implement some advanced use cases based on the earlier discussions on best practices in API design. Don't forget to take into consideration the need for optimising performance and handling errors.

Video 7: Testing RESTful API Endpoints with Postman

In the previous video, we have discussed how to develop RESTful web services using JavaScript with Express.js. Observe that we have been testing our RESTful web services using a tool known as Postman. This video will explain more about Postman to you.

Postman is a tool that allows you to send HTTP requests with different methods and various types of input and output data. Postman uses an intuitive graphical user interface that does not require you to write any code. Postman thus enables you to thoroughly test your RESTful web services before deploying them into production. Let's get ourselves oriented to the user interface of Postman.

Postman is available as a standalone application as well as a plugin for Visual Studio Code. This video will use the Postman plugin in Visual Studio Code. The figure shows a screenshot of the Postman plugin. On the left is the workspace and collection management panel. You can create one or more workspaces in Postman. Each workspace allows you to manage collections of HTTP requests, the environment and the request history. You can create one or more collections to organise your HTTP requests. For example, by application or by testing scenarios. Each collection enables you to create multiple HTTP requests. Creating on the "New HTTP Request" button will open a new request tab. In the new request tab, you can name the request or by default, Postman will use the request URL to name the request. You can save the request with the selected name in a collection. Saved requests can be easily reopened for future testing.

The HTTP request tab is divided into two main sections. The first section at the top is the request where you can configure the URL and input data. Observe that the HTTP methods drop-down list allows you to select the desired methods for the specified URL. You can also configure the input data to be sent with the request. The most commonly used input data are path and query string parameters as well as the request body. The Send button allows you to send the configured HTTP requests. After clicking the "Send" button, you would be able to observe the result in the response section. Let's send a HTTP GET request to the /product endpoint to retrieve the list of all product

records in the database. Within the response section, you would typically focus on the body of the response for the output data returned by the RESTful web service. In this example, you should be able to observe a JSON string representing an array of objects with each object representing a product record. Another important area in the response section, is the HTTP status code returned by the RESTful web service.

In this example, you should be able to observe HTTP 200 which indicates that the request has completed successfully with the data returned correctly. Another useful tool provided by Postman is the test tag in the request section. You can define various tests to determine whether the output data returned by the RESTful web service is as expected.

In this example, we define 2 tests. The first test checks whether the HTTP status code is 200. The second test checks whether the number of product objects in the array is 5. Recall that we have 5 product records in the database. If one or more tests are defined they will be automatically executed when you send the request. The test defined, will appear in the response section. Observe that the 2 tests have passed as expected. Defining tests in Postman allows you to automate the testing of your RESTful web services, which helps to ensure that your backend is working correctly and reliably. Postman contains many other useful features to aid you in the development of RESTful web services. Be sure to explore Postman further on your own!

Video 8: Creating RESTful API Endpoints with Flask and Connexion

This video explains to you how to create RESTful web services in Python.

Recall that in an earlier module, you have already learnt how to create a server-side rendering web application with Python using the Flask framework. For creating RESTful web services in Python, Connexion is one of the most popular frameworks. Connexion can be used either as a standalone application or as a middleware. The standalone Connexion application can be built using the Flask framework. In this video, we will adopt this approach, which allows you to use a single Flask application to serve both HTML content and JSON formatted data. Do remember to install Connexion using the Flask extra with this pip install connexion[flask,uvicorn,swagger-ui].

Connexion allows you to define an API endpoint using the OpenAPI or Swagger specification. This declarative approach greatly simplifies the RESTful web service development process. Let's define two endpoints with the HTTP GET method to demonstrate how to work with the Connexion framework. The first endpoint will retrieve all product records from the database. The second endpoint will retrieve one product record using the productId passed in as a path parameter. The main application source file is run.py. Observe that the FlaskApp object is from the connexion library. We can continue to define normal Flask endpoints for serving HTML content. More importantly, observe the addition of the Swagger configuration file in yaml format to our Connexion application. This file is named as openapi.yaml.

Let's examine the Swagger configuration file in greater details. You can use the online Swagger editor at this URL, to help you create and edit the file. The online editor provides real-time validation and give you detailed messages to help you correct any errors in your configuration file. Once you are sure that the configurations are correct, you can copy and paste them into the actual yaml file, in this case openapi.yaml. The server section will define the base path of the RESTful web services. In this case, to call the web service methods, you need to ensure that the path starts with /api. The HTTP request will go to Flask for handling.

In this example, the web browser on the left is showing HTML content from Flask and the web browser on the right is showing JSON data from Connexion. The paths section is used to define all the paths or endpoints in the Connexion application. In this example, we have defined two paths `/product` and `/product {product id}` using the HTTP GET method. Observe that the GET endpoint of the first path will return a JSON array of objects with each object containing the defined properties. The GET endpoint of the second path will return a JSON object with the same set of properties. This endpoint also defines an input path parameter for `productId`.

So, how does Connexion know how to process the HTTP requests that it has received? Well, observe that each endpoint will define its own `operationId`. The `operationId` will map to a designated Python function in a Python module or source file. This Python function is responsible for processing the request, for example by retrieving the required data from a database. In our example, we have two `operationIds` `product.retrieveAllProducts` and `product.retrieveProduct`.

Let's examine the `product.py` source file in greater details. Observe how the database methods are being used to execute the respective select query. The raw result from a query is a list of tuple. We need to first convert the raw result to a list of dict and then use the JSON library to convert the list of dict into JSON formatted string data. Thereafter, we can return the JSON string data to the client. Observe that other than the JSON string data, we also return an appropriate HTTP status code similar to what we have done in Express.js. The return of JSON string data and HTTP status code is done using the Flask framework response object. Please remember the importance of performing error handling in RESTful web services.

Now that you have a good understanding of the Connexion programming model, let's test the RESTful web services using Postman. Examine the series of HTTP requests sent from Postman and try to understand the corresponding response received. Finally, let's take a look at the Swagger dashboard that is automatically created for you. Observe that other than the documentation of your RESTful web services endpoints, you can also test the endpoints directly from the dashboard without even having to use Postman.

This wraps up our discussion on implementing RESTful web service in Python with Connexion. Do go ahead to explore how to create endpoints with the HTTP PUT, POST and DELETE methods so as to perform the write operations of create, update and delete.

Video 9: Module Conclusion

In this module, you have learnt essential concepts on API endpoints and RESTful web services as well as how to implement them using JavaScript and Python. Observe that the general process and techniques are similar to the development of server-side rendering web applications. However, designing and implementing good RESTful Web Services involves many other important considerations as we have discussed in the earlier videos. To fully put these good practises into action will require more advanced techniques and skills.

In this module, we are just scratching the surface of API endpoints development. API endpoints and RESTful Web Services form an important part of the overall AI solution. We will revisit RESTful Web Services in a future module on AI Models Deployment and AI Models Integration with Software Applications. Until then, please continue to learn more about web applications and web services development on your own. In particular, do explore more about the Express.js framework for JavaScript and the Connexion and Flask frameworks for Python.

I wish you a fruitful learning journey ahead for this week.

THE END